README



HelixCode - Distributed AI Development Platform

HelixCode - Distributed AI Development Platform

Version: 1.1.0
Package: `dev.helix.code`
License: MIT

Field	Value
Revision	3
Created	2025-10-31
Last modified	2026-09-03
Status	Current — release `helixcode-v1.1.0`; feature 002 in flight

Where to start, and what is in flight. This README is the entry point for all project documentation (CONST §11.4.212). Two documents carry the live state and are more current than any narrative below:

Document	What it holds
`RESUME.md`	Session-resumption record (CONST §11.4.131). §2b is the **verified** way to run the system; §2c is the current full-suite retest with its failure analysis.

Document	What it holds
specs/002-adaptive-local-model-serving/	The in-flight feature — host-capability-driven local model serving. `progress.yml` is its findings ledger.
docs/operator_decisions_20260903.md	Twelve operator decisions taken 2026-09-03, each with the options it **rejected** — read before re-opening any of them. Also records what was deliberately NOT decided.

> The Phase 1-5 feature list below describes the shipped v1.1.0 platform and is not a claim about feature 002, which is mid-execution.

HelixCode is an enterprise-grade distributed AI development platform that enables intelligent task division, work preservation, and cross-platform development workflows. Built with Go and designed for scalability, HelixCode provides a robust foundation for distributed computing with automatic checkpointing, rollback functionality, and real-time monitoring.

Key Features

Phase 1: Foundation (Completed)

- **Database Schema**: Complete PostgreSQL schema with 11 tables for distributed computing
- **Authentication System**: JWT-based auth with session management
- **Worker Management**: Distributed worker registration and health monitoring
- **Task Management**: Intelligent task division with work preservation
- **Logo Integration**: Automatic asset generation with color extraction
- **REST API**: Comprehensive HTTP API with Gin framework
- **Configuration System**: Flexible config with environment variables

Phase 2: Core Services (Completed)

- **Advanced Task Division**: Intelligent task splitting with dependency management
- **LLM Provider Integration**: Multi-provider support (Llama.cpp, Ollama, OpenAI)
- **Distributed Computing**: Work preservation with automatic checkpointing

- **MCP Protocol**: Model Context Protocol implementation
- **Advanced Reasoning**: Chain-of-thought and tree-of-thoughts reasoning
- **Multi-Channel Notifications**: Slack, Discord, Email, Telegram integration

Phase 3: Workflows (Completed)

- **Project Management**: Full project lifecycle with database persistence
- **Development Workflows**: Planning, building, testing, refactoring modes
- **Session Management**: Multi-session support with context tracking
- **Workflow Execution**: Automated workflow execution with dependencies

Phase 4: LLM Integration (Completed)

- **Hardware Detection**: Comprehensive CPU/GPU/memory analysis
- **Model Management**: Intelligent model selection based on capabilities
- **Provider Architecture**: Unified interface for all LLM providers
- **CLI Interface**: Command-line interface with interactive mode

Phase 5: Advanced Features (Completed)

- **SSH Worker Pool**: Distributed worker network with auto-installation
- **Advanced LLM Tooling**: Tool calling and reasoning API integration
- **Multi-Client Support**: REST API, CLI, Terminal UI, WebSocket
- **MCP Integration**: Full protocol support with multi-transport
- **Cross-Platform**: Linux, macOS, Windows, Aurora OS, SymphonyOS
- **Mobile Ready**: Framework for iOS and Android applications

Project Status: FULLY COMPLETE

All 5 implementation phases have been successfully completed! HelixCode is now a fully functional distributed AI development platform with enterprise-grade features including:

- **Complete Distributed Computing**: SSH-based worker networks with automatic management
- **Advanced AI Integration**: Multi-provider LLM support with tool calling and reasoning
- **Comprehensive Workflows**: Full development lifecycle automation (planning → building → testing → refactoring)
- **Multi-Platform Support**: Cross-platform compatibility with mobile frameworks ready
- **Enterprise Features**: Authentication, notifications, MCP protocol, and robust APIs

Architecture

```
HelixCode Architecture
├── API Layer (REST + WebSocket + MCP)
├── Core Services
│   ├── Authentication & Session Management
│   ├── Worker Pool Management (SSH-based)
│   ├── Task Management & Checkpointing
│   ├── Project & Workflow Management
│   └── LLM Provider Integration
├── Database Layer (PostgreSQL + Redis)
├── Distributed Workers (Cross-platform)
└── Multi-Client Interfaces (CLI, TUI, REST, Mobile)
```

Project Structure

```
helix_code/
├── specification/          # Technical specifications and requirements
├── implementation_guide/   # Implementation plans and guides
├── Design/                 # Design assets and specifications
├── helix_code/              # Main Go implementation
│   ├── cmd/
│   │   ├── server/         # HTTP server application
│   │   └── cli/            # CLI client
│   ├── internal/
│   │   ├── auth/           # Authentication system
│   │   ├── config/         # Configuration management
│   │   ├── database/       # Database layer
│   │   ├── hardware/       # Hardware detection
│   │   ├── llm/            # LLM providers and reasoning
│   │   ├── logo/           # Logo processing & assets
│   │   ├── mcp/            # MCP protocol implementation
│   │   ├── notification/   # Multi-channel notifications
│   │   ├── project/        # Project management
│   │   ├── server/         # HTTP server & API
│   │   ├── session/        # Session management
│   │   ├── task/           # Task management & checkpoints
│   │   ├── worker/         # Worker pool management
│   │   └── workflow/       # Workflow execution
│   └── scripts/            # Build and utility scripts
├── website/                # Marketing website
├── assets/                 # Project assets and logos
└── docs/                   # Documentation
```

Quick Start

Prerequisites

- Go 1.26+ (the inner `helix_code/` module declares `go 1.26`; the meta-repo root is `go 1.25.2`)
- PostgreSQL 15+
- Redis 7+ (optional)

Installation

1. **Clone the repository and setup environment**:

   ```
   git clone git@github.com:HelixDevelopment/HelixCode.git
   cd HelixCode
   ./setup.sh
   ```

   This will:

   - Initialize all git submodules
   - Install system dependencies
   - Build the HelixCode application

2. **Manual setup (alternative)**:

   ```
   # Initialize submodules
   ./scripts/init-submodules.sh

   # Install dependencies (Ubuntu/Debian)
   ./install_missing_libs.sh

   # Build the application (inner Go module — lowercase per
           CONST-052)
   cd helix_code
   make build
   ```

3. **Put the HelixCode power sub-systems on your PATH (optional, no sudo)**:

   ```
   ./install_helix_path.sh
   ```

   Builds (or reuses already-built) `helixcode`, `helixcode` CLI, `helixagent`, `helixllm`, `llms-verifier`, and `helixqa`, symlinks each into `$HOME/.local/bin` (override with `HELIX_BIN_DIR`), and idempotently appends the PATH export to your shell rc. Re-run any time — it never double-installs or duplicates the PATH block. See the script's own header comment for full usage, and `tests/install_helix_path/test_install_helix_path.sh` for its black-box test suite.

4. **Configure the required secrets** — the server REFUSES to start without them, naming the missing variable rather than falling back to a literal:

```
cp .env.example .env && chmod 600 .env
# then set these three to real values (.env is gitignored):
#   HELIX_DATABASE_PASSWORD
#   HELIX_REDIS_PASSWORD
#   HELIX_AUTH_JWT_SECRET
# e.g. for a local run:  openssl rand -hex 32
```

5. **Run the server**:

```
cd helix_code
set -a; . ../.env; set +a
HELIX_REDIS_HOST=localhost ./bin/helixcode
```

You do NOT need to create a database. On startup the server boots its own PostgreSQL and Redis containers on demand (per the Containers submodule, CONST §11.4.76) and repoints its own config at them, so a fresh run works with no `createdb`, no compose file and no manual container commands:

```
Infra auto-boot: podman booted postgres:<port> redis:<port>
Database connection established successfully
Starting HelixCode server on 0.0.0.0:8080
```

Two containers appear (`helixcode-autoboot-postgres`, `helixcode-autoboot-redis`). Set `HELIX_AUTOBOOT_INFRA=false` to use externally-provisioned infrastructure instead.

> **Container runtime.** This project mandates **rootless podman** (CONST §11.4.161); `docker` is not required and is not used. The `./helix` facade resolves podman first and falls back to docker only if podman is absent.

6. **Verify it is serving**:

```
curl -s http://localhost:8080/health
# {"status":"healthy","version":"1.0.0"}
curl -s http://localhost:8080/api/v1/llm/providers
```

CLI Usage

```
# Interactive mode
./cli

# List workers
./cli --list-workers

# Add a worker
```

```
./cli --worker worker-host --user helix --key ~/.ssh/id_rsa

# Generate with LLM
./cli --prompt "Hello world" --model llama-3-8b

# Health check
./cli --health
```

API Endpoints

Authentication

- `POST /api/v1/auth/register` - User registration
- `POST /api/v1/auth/login` - User login
- `POST /api/v1/auth/refresh` - Token refresh

Workers

- `GET /api/v1/workers` - List workers
- `POST /api/v1/workers` - Register worker
- `GET /api/v1/workers/:id` - Get worker details

Tasks

- `GET /api/v1/tasks` - List tasks
- `POST /api/v1/tasks` - Create task
- `GET /api/v1/tasks/:id` - Get task details
- `POST /api/v1/tasks/:id/start` - Start task execution

Projects

- `GET /api/v1/projects` - List projects
- `POST /api/v1/projects` - Create project
- `GET /api/v1/projects/:id` - Get project details

Database Schema

Core Tables

- **users**: User accounts and authentication
- **workers**: Distributed worker nodes with SSH config
- **tasks**: Task management with checkpoints and dependencies
- **projects**: Project lifecycle management
- **sessions**: Development sessions and context

- **llm_providers**: Configured LLM provider instances
- **notifications**: Multi-channel notification management

Development

Build Commands

```
make build          # Build the application
make test           # Run all tests
make clean          # Clean build artifacts
make lint           # Lint code
make fmt            # Format code
```

Testing

```
# Run all tests
go test ./...

# Run specific package tests
go test -v ./internal/auth

# Run with coverage
go test -cover ./...
```

Contributing

1. Fork the repository
2. Create a feature branch (`git checkout -b feature/amazing-feature`)
3. Commit your changes (`git commit -m 'Add amazing feature'`)
4. Push to the branch (`git push origin feature/amazing-feature`)
5. Open a Pull Request

License

This project is licensed under the MIT License - see the LICENSE file for details.

Documentation

- Architecture Overview
- Development Guide
- User Guide
- API Documentation

Skills System (HXC-159)

- Capabilities — Skills sections §4/§10.5
- Skills Architecture (three-layer diagram + registration path)
- Skills User Manual
- Skills FAQ
- Extensions Catalog (per-platform compatibility matrix, §11.4.228)
- HelixAgent Skills System (submodule-side capability-enforcement model)
- Constitution-Powered SpecKit-Superpowers Bridge (design corpus)

Support

- **Issues**: tracked in-repo at `docs/Issues.md` (+ `docs/Fixed.md`), the SQLite-backed single source of truth `docs/workable_items.db` (§11.4.93/95)
- **Documentation**: See the Documentation section above + `docs/`

Built with using Go, PostgreSQL, and distributed computing principles

HelixCode - Empowering distributed AI development workflows

Sources verified

Per constitution §11.4.99 (Latest-Source Documentation Cross-Reference), the operator-facing instructions in this README were cross-referenced against the latest official sources and the repository's actual state on **2026-05-29**:

- **Go version** — README now requires **Go 1.26**+, matching `helix_code/go.mod` (`go 1.26`) and the meta-repo root `go.mod` (`go 1.25.2`). Verified against the official Go release history (https://go.dev/doc/devel/release): Go 1.26.0 released 2026-02-10 (latest 1.26.3, 2026-05-07); Go 1.24 is now past its support window. The prior "Go 1.24.0+" was incorrect — the inner module does not build below Go 1.26.

- **Clone URL** — corrected to the real SSH remote `git@github.com:HelixDevelopment/HelixCode.git` (was a `https://…/your-org/…` placeholder), satisfying the SSH-only Git rule (Constitution Rule 3 / CONST-038, GitHub+GitLab only).

- **Inner-module path** — corrected `cd HelixCode` → `cd helix_code` (lowercase per CONST-052; verified the on-disk directory is `helix_code/`).

- **Referenced scripts** — `setup.sh`, `scripts/init-submodules.sh`, `install_missing_libs.sh` all confirmed present in the tree.

- **PostgreSQL 15+ / Redis 7+** — consistent with the authoritative tech stack in `CLAUDE.md` §3.1 (pgx/v5 on PostgreSQL 15+, go-redis/v9 on Redis 7+); both minimums remain valid against current upstream (PostgreSQL 17 / Redis 7.x are the current GA lines, backward-compatible).

- **Release sync (2026-06-24)** — the `**Version**` field was bumped `1.0.0` → `1.1.0` to track the `helixcode-v1.1.0` release tag (minor bump over `helixcode-v1.0.0`); verified against `git tag -l 'helixcode-v*'` (only `helixcode-v1.0.0` and `helixcode-v1.1.0` exist; `v1.1.0` tagged 2026-06-24).

- **Metadata table added (2026-06-24)** — the prior negative finding (legacy §11.4.44 bold-line header used instead of the §11.4.61/§CONST-064 metadata table) is resolved: the four mandatory rows (Revision/Created/Last modified/Status) are now present. `Created` (2025-10-31) is sourced from the README's first commit (`git log --reverse -- README.md`), not guessed.

- **Quick Start corrected against a real run (2026-09-03)** — the previous steps were verified by actually executing them, and three were wrong:

  - *"Setup database: `createdb helixcode` / `createuser helixcode`"* was **removed**. It is not merely unnecessary, it describes the wrong architecture: on startup the server calls `infraboot.EnsureInfra` and boots its OWN PostgreSQL + Redis containers, then repoints its config at them (`cmd/server/main.go`, per CONST §11.4.76 on-demand-infra). Observed: `Infra auto-boot: podman booted postgres:55432 redis:56379`, followed by a successful connection and schema creation, with `helixcode-autoboot-{postgres,redis}` both healthy. `HELIX_AUTOBOOT_INFRA=false` opts out.
  - *"Run the server: `./bin/helixcode`"* omitted the environment. The server REFUSES to start without the secrets, by design, naming the variable: `auth.jwt_secret is still the unexpanded placeholder for ${HELIX_AUTH_JWT_SECRET}`. The three required variables and the `.env` route are now stated.
  - The **container runtime** is now stated explicitly. This host has podman and no docker at all, which is what CONST §11.4.161 requires; the `./helix` facade was podman-blind until `d5158bd3` and exited 1 on every command with *"Docker is not installed or not in PATH"*.
  - A **verification step** was added (`/health`, `/api/v1/llm/providers`), both confirmed 200.

- **Duplicate section removed (2026-09-03)** — the "Phase 4: LLM Integration" block appeared TWICE in Key Features (once before Phase 3 and once after), leaving the phase list ordered 1, 2, 4, 3, 4, 5. The first copy was removed; the order is now 1-5.

- **Live-state pointers added (2026-09-03)** — per §11.4.212 (README is the canonical entry point) the header now routes readers to `RESUME.md` and the in-flight feature 002 directory, and states plainly that the Phase 1-5 list describes shipped v1.1.0 rather than feature 002.

Re-verify before the next release boundary or if any tool above ships a breaking change (§11.4.99(C) — instructions older than 6 months are stale).

Sources verified 2026-06-24: https://go.dev/doc/devel/release ; repo cross-reference (`helix_code/go.mod`, `go.mod`, on-disk paths, `git remote`, `CLAUDE.md` §3.1, `git tag -l`, `git log -- README.md`).

Sources verified 2026-09-03: verified by EXECUTION rather than by reading — the server was built and run, its auto-boot and refusal paths observed in its own log, the endpoints probed, and the podman-only runtime confirmed on the host (`command -v docker` absent, `command -v podman` present). Repo cross-reference: `helix_code/cmd/server/main.go` (`infraboot.EnsureInfra`), `docker-compose.helix.yml`, `helix`, `RESUME.md` §2b/§2c.